

흥미로운 ux 를 제공하면서도 성능 최적화를 통해 불편함 없는 사용자 경험을 만들어가는데 관심이 많은 개발자입니다.

- 디자인 역량을 바탕으로, 스스로 UI/UX 디자인을 제작하고 이를 다양한 CSS 기법으로 구현하여 개발까지 완성해 나가고 있습니다.
- 교육학에서 배운 사용자 중심 사고와 문제 해결 능력을 이용하여 사용자 경험을 중심으로 문제를 분석하고 해결책을 찾아 문제를 해결해 나가고 있습니다.
- 디자인과 교육학을 통해 배운 효과적인 의사소통 능력을 바탕으로 피드백을 수용하고 프로젝트를 발전시키며, 문제점을 발견하고 해결하는 데 강점을 발휘하고 있습니다

기술스택

프론트엔드 기술

핵심 기술: React, TypeScript
 프레임워크: Next.js, Remix
 상태 관리: Redux, Zustand
 서버 상태 관리: Tanstack Query
 라우팅: Tanstack Router
 애니메이션 및 3D: GSAP, React Spring, Three.js
 스타일링: Styled-Components, TailwindCSS
 폼 검증 및 타입 체크: Zod, Valibot
 테스트: Jest, React Testing Library

백엔드 기술

런타임: Node.js
 프레임워크: Express
 데이터베이스: MySQL, PostgreSQL
 ORM: Prisma

개발 환경 및 도구

CI/CD: GitHub Actions
 Git Hooks: Husky
 코드 품질 관리: ESLint, Prettier

프로젝트

학습 내용을 정리하고 흥미로운 디자인을 이용하여 블로그

sun_ub 블로그

2023.06~

[sunub/sunub.github.io](https://github.com/sunub)

sunub.vercel.app

디자인 : Figma, Photoshop

코어 : React, TypeScript, Next.js,

MDX

ORM: Prisma

데이터베이스 : Postgresql

스타일링 : Styled-component,

GSAP

- 문제** 페이지 진입 시 고화질 이미지와 다양한 애니메이션으로 인한 초기 로딩 지연
- 분석** Lighthouse와 Chrome DevTools로 네트워크 병목과 이미지 최적화 필요성 확인
- 해결**
 - 고화질의 히어로 이미지를 이미지 포맷을 Avif로 변경하고 크기를 감소 시켜 이미지 최적화
 - 폰트의 크기를 줄이기 위하여 ttf에서 woff2로 포맷을 변경하여 폰트 최적화
 - CSS를 활용하여 GPU 가속(transform, opacity) 및 Critical CSS로 렌더링 속도 개선
- 결과** LCP 시간을 3.4s에서 1.0s로 개선하여 페이지 이탈률 감소 및 사용자 경험 개선
- 문제** 초기 페이지 진입 후 포스트로 이동할 경우 페이지 로딩 지연
- 분석** DevTool의 Performance를 이용하여 네트워크와 메인 스레드에서 작업 과부하 확인
- 해결**
 - 반복되는 처리하는 IO작업을 캐싱 및 병렬로 처리하여 JavaScript 실행 대기 시간(idle)을 3097ms에서 854ms로 단축
 - SSR과 SSG를 이용하여 초기 페이지 로드 시간 단축
- 결과** 페이지 이동 성능 시간을 3.5s 에서 889ms 로 개선하여 페이지 이탈률 감소 및 사용자 경험 개선
- 문제** 디바이스별 다양한 화면 크기에서 레이아웃이 어색하게 표시되거나, 요소 크기가 잘리지 않아야 할 정보가 잘리는 현상이 발생.
- 분석** 기존 미디어 쿼리는 뷰포트 크기에 의존해, 동일한 컨테이너에서 다른 크기의 컴포넌트가 적응하지 못하는 문제를 발견
- 해결**
 - CSS @container 규칙을 적용하여 각 컨테이너에 유연한 레이아웃 정의.
 - 컴포넌트 크기와 글자 크기가 자동 조정되도록 디자인을 개선하여 모바일 및 태블릿 사용자 경험 최적화
- 결과** 디바이스에서의 접근성을 개선하여 방문자 수 증가 가능성 증진

프로젝트

betting-duck 프로젝트

2024.10 ~ 2024.12

FE 2명, BE 2명 중 FE 담당

[boostcamp/betting-duck](https://github.com/boostcamp/betting-duck)

[betting-duck-wiki](https://github.com/betting-duck/wiki)

betting-duck.org

디자인 : Figma

코어 : React, TypeScript

스타일링 : Tailwindcss

서버 상태 관리: Tanstack Query

라우팅: Tanstack Router

폼 검증 및 타입 체크: Zod

빌드/번들링 툴 : Vite

3D 렌더링 : Three.js

실시간 통신 : Socket.io

실시간 베팅과 채팅에 참여하고, 획득한 코인으로 마이페이지를 꾸밀 수 있는 서비스입니다

- 문제** 프록시 서버와의 통신으로 네트워크 지연이 발생, API 응답 타이밍이 어긋나 상태 변화가 제대로 반영되지 않거나 UI가 중단되는 오류 발생
- 분석**
- Postman을 이용해 프록시 서버를 통한 요청과 직접 백엔드와 통신하는 요청의 응답 시간을 비교.
 - 서버 로그와 웹 네트워크 요청(Request/Response) 데이터를 대조하여 API 응답 타이밍과 상태 변화를 추적

- 해결**
- tanstack query의 자동 재시도 매커니즘 활용하여 응답에 실패하였을 경우 다시 요청을 전송
 - 기존의 묶여 있던 상태 변경 API 호출과 페이지 리다이렉트 로직을 두 개의 로직으로 분리하여 관리
 - API가 지연되어 응답될 경우 사용자에게 toast 메시지를 이용하여 피드백 제공

- 결과** API 실패율 감소 및 페이지 중단 문제 해결을 통해 사용자 경험 개선

문제 Socket.IO 기반 실시간 데이터 송수신 구조 설계 및 구현

- 해결**
- Socket.IO를 활용하여 실시간 데이터 송수신 시스템 설계 및 구현
 - 커스텀 훅을 활용하여 연결 수립 및 네임스페이스 기반 소켓 이벤트 관리로 각각의 소켓간의 독립성을 유지
 - 페이지 전환 시 기존 연결 해제 및 새로운 연결 설정으로 안정성 강화

- 결과** 연결 불안정성 및 중복 연결 문제를 해결

문제 API 통신 시 데이터 구조 불일치로 인하여 개발 과정에서의 불필요한 의사소통 및 예기치 못한 런타임 에러가 발생

- 분석** DevTools을 활용해 런타임 에러 로그를 분석

- 해결**
- 패키지 매니저 중 다른 패키지 매니저들 중 내장된 워크스페이스 기능을 통해 프로젝트 간 의존성을 쉽게 연결하고 관리할 수 있는 pnpm 사용
 - 프론트엔드와 백엔드가 공통으로 사용할 수 있는 타입 정의 패키지를 모노레포 내에 생성
 - Zod로 API 요청/응답 스키마를 정의하여 정적 타입(TypeScript)과 런타임 검증을 동시에 제공
 - 동일한 타입을 프론트엔드와 백엔드에서 공유해 데이터 구조 불일치 문제 해결

- 결과** 의사소통 비용 감소와 더불어, API 요청/응답 데이터의 신뢰성 향상

프로젝트

for-digital-divide 프로젝트

2023.11 ~ 2024.05

[sunub/for-digital-divide](https://github.com/sunub/for-digital-divide)

for-digital-divide

디자인 : Figma, Photoshop

코어 : React, TypeScript, Next.js

유효성 검사 : Zod

상태 관리 : Zustand

데이터베이스 : PostgreSQL

스타일링 : Styled-component,

Tailwind-css, framer-motion

PIN을 이용하여 로그인하는 보안 프로그램을 만들어보는 프로젝트

- 문제** PIN 번호 입력 시, input 필드를 통해 입력값이 키보드 입력 및 화면 녹화와 같은 공격에 노출될 수 있는 보안 문제 발생

- 해결**
- CSS와 Next.js를 활용해 PIN 패드 버튼의 이미지 위치를 무작위로 변경, PIN 번호 입력 시 화면 정보를 유출하기 어렵게 구현.
 - Next.js의 Route Handlers를 사용하여 PIN 패드의 버튼 위치 정보를 매 요청마다 랜덤화해 클라이언트에 전달
 - 사용자가 PIN 패드의 버튼을 클릭할 때, 마우스 클릭 좌표와 서버에서 제공된 버튼의 랜덤 좌표를 비교하여 PIN 입력을 처리

- 결과** 사용자 PIN 정보가 클라이언트에 저장되지 않아 데이터 유출 위험 감소.

여러 인터랙션 및 애니메이션

2023.11 ~ 2024.05

 [css-canvas-playground](#)

 [webgl-canvas-playground](#)

디자인 : Figma, Photoshop

코어 : React,

스타일링 : Styled-component,

Three.js, gsap,

빌드 : Vite

교육

부스트캠프 웹-모바일 9기

(2024.06 ~ 2024.12)

풀스택 과정

Kent C. Dodds의 EpicShop

(2024.02 ~ 2024.06)

Full Stack Foundations,
Authentication Strategies,
Professional Web Forms,
Data Modeling Deep Dive

JoshW. Comeau의 Joy Of React

(2023.09 ~ 2023.12)

SVG, CSS, Three.js 등을 활용하여 애니메이션 작업 및 장면을 구성한 프로젝트

학습

- Three.js를 활용해 로컬 파일 업로드 및 GLTF/OBJ 형식의 3D 모델을 렌더링하고, 카메라 조작(회전, 줌, 이동) 인터랙션 구현
- transform, opacity, clip-path 등을 사용한 고성능 애니메이션 개발
- GPU 가속 애니메이션을 통해 부드러운 동작 구현 (e.g., transform 및 opacity 중심의 애니메이션)

성과

- CSS Keyframes 및 Transitions와 GSAP을 비교하여 적합한 사용 사례를 이해하고 적용
- Three.js의 장단점 및 실제 프로젝트에 적용 시 예상되는 리소스 요구사항에 대한 실무적 통찰 획득

학습

- Git, 프로세스 메모리 시뮬레이터 등을 직접 구현하며 CS 지식과 CS 지식이 어떻게 활용되고 있고 어떻게 활용하는지에 대해 학습
- 함께 성장하는 것의 중요성과 함께 성장하기 위해서 지식을 공유하는 방법, 피드백을 주고 받는 방법에 대해 학습
- Node.js의 이벤트 기반 아키텍처와 비동기 처리와 이벤트 루프에 대하여 학습

성과

- Node.js의 이벤트 루프와 비동기 처리를 이해하여 서버-클라이언트 간 데이터 처리에서 성능 최적화를 효과적으로 적용
- 학습한 내용을 체계적으로 문서화하며 팀원들과의 지식 공유가 원활해졌고, 팀의 성장에 기여

학습

- CSS 관리, 라우팅, 데이터 로딩 등 최신 모범 사례를 학습하여 빠르고 반응성 높은 웹 애플리케이션을 구축할 수 있는 능력 향상
- Prisma와 SQLite를 사용하여 견고한 데이터베이스 스키마를 설계하고, 마이그레이션 및 시딩 데이터를 효과적으로 관리하는 방법을 학습
- 폼 검증 및 보안을 강화하고, 접근성과 사용자 경험을 향상시켰으며, 복잡한 데이터 처리와 성능 최적화에 대해 학습
- 컴포넌트 API 디자인, 접근성 고려, 성능 최적화 등을 통해 재사용 가능하고 효율적인 컴포넌트를 설계하는 방법에 대해 학습

성과

- React의 기본 원리부터 고급 개념까지 탄탄한 이론적 기반과 실무적 적용 능력을 습득
- PostCSS와 Tailwind CSS를 사용하여 효율적인 스타일링과 성능 최적화 달성
- 폼 검증 및 보안 강화 기술의 심화 학습과 실전 구현 역량 확보
- 데이터 모델링 및 데이터베이스 최적화에 대한 전문 지식과 실무 적용 능력 배양